

Lab2 Mini-report

Code Handling Practices and Lexer Features

How you handle comments

注释最难的部分是Tiger语言的注释是可以嵌套的（即 `/* ... /* ... */ ... */`），而标准正则表达式本质上只能处理正则文法，无法实现“括号匹配”这种上下文无关文法的需求。

为了处理嵌套注释，我使用了 **计数器**：

1. **状态转换**：在 `INITIAL` 状态下遇到 `/*` 时，触发进入 `%x COMMENT` 独占状态，并将计数器 `comment_level_` 初始化为 1。
2. **嵌套层级**：在 `<COMMENT>` 状态内，每遇到一个完整的 `/*`，`comment_level_` 加 1；每遇到一个 `*/`，`comment_level_` 减 1。
3. **退出注释**：只有当 `comment_level_` 归零时，说明最外层注释也已闭合，此时调用 `begin(StartCondition_::INITIAL)` 回归正常的词法分析状态。
4. **位置维护**：在注释中完全忽略普通字符（不返回 Token），同时利用 `\n` 规则调用 `errorMsg->Newline()`，确保跨越多行的注释不会导致后续代码行号计算错乱。

How you handle strings

Tiger 的字符串包含各种复杂的转义及长文本排版语法。我主要通过 `%x STR` 和 `%x IGNORE` 两个独占状态联合 `string_buf_` 缓冲区来完成：

1. **缓冲与状态隔离**：遇到左侧 `"` 时**清空** `string_buf_` 并进入 `<STR>`。所有普通字符会被追加进 `string_buf_`，直到遇到未被转义的右侧 `"`，再通过 `setMatched(string_buf_)` 将真实的去除转义后的字符串上报给语法分析器。
2. **转义字符处理**：
 - 基础转义：直接将 `\n`，`\t`，`\\`，`\"` 映射为 C++ 对应的普通转义字符推入缓冲区。
 - ASCII 八进制/十进制（`\ddd`）：提取数字后用 `std::stoi` 转换类型，并校验是否超过 255。
 - 控制字符（`\^c`）：利用 ASCII 表的特性，通过 `matched()[2] - '@'` 将对应英文字母转为特定的机器控制码。
3. **多行字符串格式化（`\ ... \`）**：当在 `<STR>` 中遇到单反斜杠 `\` 时，跨入 `<IGNORE>` 状

态。在该状态下，利用 `[\t\n\f]` 消耗所有的空白和换行，直至遇到下一个 `\` 再返回 `<STR>`，从而舍弃了排版用的多余换行与缩进。

Error handling

我在词法分析器的各个部分加入了细致的报错捕获（结合 `errmsg->Error()` 并附带准确的坐标）：

1. **非法字符过滤**：在 `INITIAL` 底部使用 `.` 捕获所有不属于任何 Token 的游离字符并抛出 "illegal token"。
2. **未闭合字符串拦截**：在 `<STR>` 状态中增加对独立的 `\n`（真实的换行符）的拦截以抛出 "newline in string" 错误，并强行阻断当前匹配退回 `INITIAL`。
3. **忽略块非法阻断**：在 `<IGNORE>` 状态下加入了 `.` 捕获。若两个 `\` 之间出现了非空字符，则抛出 "illegal character in ignore sequence"，严格限制多行忽略机制只针对空白符。
4. **越界捕捉**：对形如 `\999` 的错误给出 "ASCII out of range" 警告。
5. **文件末尾处理**：在词法分析器中加入 `<COMMENT><<EOF>>` 和 `<STR><<EOF>>` 规则，从而在文件末尾时能正确地抛出 "unclosed comment/string" 错误。

End-of-file handling

- **正常情况**下的到达文件末尾（**INITIAL 状态**）
 - 当代码处于正常的 `INITIAL` 状态（没有在解析字符串，也没有在解析注释），在顺利读完最后一个合法字符后，Flex 引擎会自然地撞到文件末尾。默认情况下，Flex++ 引擎包含了一条隐式的保底规则，当遇到文件末尾且没有挂起的独占状态时，`lex()` 函数会自动返回 0。
- **异常情况**下的到达文件末尾
 - 如果代码忘记闭合注释或字符串，会导致分析器卡死在对应的独占状态中直到触碰 EOF 崩溃。通过增加特殊的词法规则 `<COMMENT><<EOF>>` 和 `<STR><<EOF>>`，我的词法分析器可以安全地拦截到文件意外结束的情况，并调用 `errmsg->Error(..., "unclosed comment/string")` 抛出准确的忘写闭合符的提示，最后直接退出（`return 0`）。

Other interesting features of your lexer

区分 `adjust()` 与 `adjustStr()` 进行精准定位：

为了向编译器前端提供准确的报错坐标，我利用了提供的游标机制。对于独立的 Token（关键

字、符号)，我使用 `adjust()` 同时更新游标总推进量 `char_pos_` 和 Token 起始锚点 `tok_pos_`。而对于需要状态机多次游历的 String 解析过程，我在内部统一调用 `adjustStr()`，这样只推进总游标而**不覆盖**最初双引号处的起点坐标 `tok_pos_`，这保证了无论字符串多长，编译器报错时指向的永远是字符串开头的位置。

Challenges and Bug Fixes

Challenges during your implementation

1. 关键字与标识符 (Keyword vs. ID) 的优先级处理：

在最初编写过程中，我遇到了如何让词法分析器分清 `if`（关键字）和 `iffy`（普通变量名）以及相同长度词汇冲突的挑战。通过词法分析器的“**最长匹配原则**”和“**靠前匹配规则**”，我将保留关键字（如 `if`, `while`, `array`）的声明严格放置在了标识符 `{letter}({letter}|{digit}|_)*` 的上方，从而顺利解决了关键字处理混乱的问题。

2. Tiger 多行字符串忽略机制的状态挂载：

Tiger `\ [whitespace] \` 这种消耗连续多行多余空格的特性很难在一个状态内完成，我采用了“状态套状态”的方法，最终成功通过 `INITIAL -> STR -> IGNORE` 的三级状态实现了字符串内容的处理。

Bug fix log

• Bug 1: 逻辑运算符匹配错误

- **Description:** 最开始依照直觉写了 `"and"` 和 `"or"` 来匹配与和或运算符。
- **Fix:** 经过仔细核对 Tiger 语言规范后，将规则修正为 Tiger 的实际逻辑操作符 `"&"` 和 `"|"`，对应的 Token 为 `Parser::AND` 和 `Parser::OR`。

• Bug 2: 换行导致的匹配穿透缺失

- **Description:** 发现在字符串 (`<STR>`) 内如果不写右侧双引号且直接敲击回车，遇到 `\n` 时，由于正则的点号 `.` 默认不包含换行符，分析器会发生底层未匹配而崩掉，未能输出自定义报错。
- **Fix:** 为 `<STR>` 单独加入了对 `\n` 换行符的捕获规则，使用 `errmsg_ -> Error` 给出报错提示，并强制 `begin(StartCondition_::INITIAL)` 使其从错误中恢复到稳态恢复，提升了 Lexer 的稳定性。

• Bug 3: 没有考虑代码忘记闭合注释或字符串的情况

- **Description:** 在 `<COMMENT>` 和 `<STR>` 状态中, 如果代码忘记闭合注释或字符串, 会导致分析器卡死在对应的独占状态中直到触碰 EOF 崩溃。
- **Fix:** 看到mini-report要求中的end-of-file部分发现需要处理这一部分内容, 通过增加特殊的词法规则 `<COMMENT><<EOF>>` 和 `<STR><<EOF>>` 解决了这一问题。